To bind or not to bind? Replaying Object-centric Processes under stable Relationships

No Author Given

No Institute Given

Abstract. Object-centric event logs record the intertwined behavior of multiple business objects. To replay these event logs, object-centric Petri nets (OCPN) have been introduced. While OCPNs indicate how objects of different types flow through the process, and how they co-participate in process activities establishing stable many-to-one relationships, they do not provide mechanisms for tracking the identity of objects and their mutual relationships. On the one hand, this makes it ambiguous to understand how objects should synchronize, an ambiguity that is resolved in models providing explicit support for object identifiers, such as most prominently object-centric Petri nets with identifiers (OPIDs). On the other hand, OCPNs often informally appear "just fine" in intuitively capturing the intended process semantics for a large number of object-centric executions, whose characteristics have never been made explicit. In this paper, we aim to close this gap rigorously through the formal identification of a class of object-centric executions that yields a non-ambiguous interpretation of the execution and synchronization semantics of a given OCPN. We then show that every OCPN can be unambiguously encoded into a corresponding OPID where objects, stable relationships, and synchronization points are made explicit so that the two specifications coincide when focussing on executions with stable relationships. We comment on the impact of this result on process modeling, analysis, and mining.

Keywords: Business Process Management · Conformance Checking · Object-centric Petri Nets · Object-centric Event Logs · Synchronization.

1 Introduction

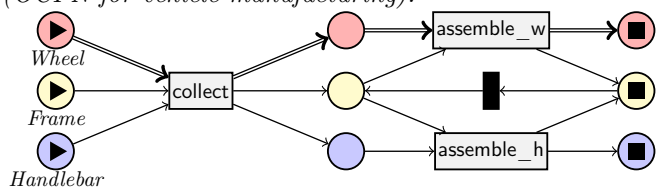
Process mining analyzes behavior in information systems through event logs to gain insights into the system's behavior [13]. Conformance checking identifies which executions in an event log conform to a given process model [3]. A process model can serve as a trace recognizer that replays executions to accept conform traces and reject non-conform traces. The novel research area of object-centric process mining focuses on the combined behavioral perspectives of multiple objects [12]. Object-centric event logs (OCEL) record objects of specific types and how events operate on these objects [4]. For example, consider the following OCEL L_1 that represents the behavior in a bicycle manufacturing workshop. For a given order, a frame, a set of wheels, and a handlebar are collected before being assembled. An important feature of this log is that the frame, its

wheels, and its handlebar are always processed in stable relationships such that the objects form stable links, e.g., (f_1, h_1), (f_1, w_1), and (f_1, w_2).

$$L_1 = [\ \langle \mathsf{collect}, \{f_1, h_1, w_1, w_2\}\rangle, \langle \mathsf{assemble_w}, \{f_1, w_1, w_2\}\rangle, \langle \mathsf{assemble_h}, \{f_1, h_1\}\rangle, \langle \mathsf{collect}, \{f_2, h_2, w_3, w_4\}\rangle, \langle \mathsf{assemble_h}, \{f_2, h_2\}\rangle, \langle \mathsf{assemble_w}, \{f_2, w_3, w_4\}\rangle,\ ]$$

To replay the behavior of such OCELs, object-centric Petri nets (OCPN) can be discovered [14]. For example, the OCPN in Example 1 allows for replaying the behavior of log L_1. In the initial marking (▶), all recorded objects of the log are placed. The transitions can fire by binding objects and processing them through the net. A non-variable arc binds a single object, while a variable arc (double line) binds a non-empty set of objects. Finally, the net accepts a given execution if all objects are placed in the final marking (■).

Example 1 (OCPN for vehicle manufacturing).



While OCPNs have been studied for conformance checking [1], an important limitation has not yet been discussed in detail. OCPNs can not represent links between objects and are, therefore, lacking synchronization behavior [5]. Therefore, the example OCPN accepts the execution in the following log L_2 even though the expected stable relationships are violated, e.g., $\langle \mathsf{collect}, \{f_3, h_3, w_5, w_6\}\rangle$ might expect the link (f_3, w_6) but $\langle \mathsf{assemble_w}, \{f_4, w_6, w_7\}\rangle$ violates it. On the other hand, the recent formalism of object-centric Petri nets with identifiers (OPID) [5] supports object identifiers, links, and synchronization.

$$L_2 = [\langle \mathsf{collect}, \{f_3, h_3, w_5, w_6\}\rangle, \langle \mathsf{collect}, \{f_4, h_4, w_7\}\rangle, \langle \mathsf{assemble_w}, \{f_3, w_5\}\rangle, \langle \mathsf{assemble_h}, \{f_3, h_4\}\rangle, \langle \mathsf{assemble_w}, \{f_4, w_6, w_7\}\rangle, \langle \mathsf{assemble_h}, \{f_4, h_3\}\rangle]$$

Therefore, we define a mapping from OCPNs to OPIDs, which implements stable many-to-one relationships. We do so in four steps. First, OCPNs are mapped to equivalent OPIDs with the same ambiguity in replaying object relationships. Second, we introduce *stable many-to-one relationships* between object types, where whenever two objects of those types co-participate in an event, they form a stable relationship that is never altered. Third, we provide an encoding that, given an input OCPN and stable relationships, produces a corresponding OPID in which objects are explicitly synchronized according to their stable relationships. Finally, we show that, while the so-obtained OPID is, in general, more restrictive than the original OCPN, the two specifications are indeed identical when restricting only to executions with stable relationships. Despite their ambiguity, we can give the formal guarantee that OCPNs are precise enough to replay executions that conform to the stable many-to-one relations.

The remainder of this paper is structured as follows. Section 2 provides the preliminary definitions from related work. With this basis, Section 3 defines an equivalent mapping from OCPN to OPID, which is extended by Section 4 to implement stable many-to-one relations. Section 5 discusses this contribution in the context of related work before Section 6 concludes the paper.

2 Preliminary Definitions

As the basis to our formal investigation, we refer to the existing definitions for OCEL, OCPN, and OPID definitions in the following.

2.1 Object-centric event logs and object-centric Petri nets

To define OCELs and OCPNs, we refer to the notion by van der Aalst and Berti [14]. Both are defined for a fixed set of object types Σ and activities $\mathcal{A}$.

Definition 1. *Given a set of object types Σ, a set of activities $\mathcal{A}$, and a set of timestamps $\mathbb{T}$ under a total order $<$, an* object-centric event log *(OCEL) is a tuple $L = (E, \mathcal{O}, \pi_{act}, \pi_{obj}, \pi_{time})$ where:*

- *E is a set of events;*
- *$\mathcal{O}$ is a set of object identifiers, each typed by the function $ot : \mathcal{O} \to \Sigma$;*
- *the functions $\pi_{act} : E \to \mathcal{A}$, $\pi_{obj} : E \to \mathcal{P}(\mathcal{O})$, and $\pi_{time} : E \to \mathbb{T}$, mapping each event to an activity, a set of objects, and a timestamp.*

For example, the event logs L_1 and L_2 in Section 1 are defined for the object types *Wheel*, *Frame*, and *Handlebar*. The event $e = \langle \mathsf{collect}, \{f_1, h_1, w_1, w_2\}\rangle$ in L_1 refers to the activity $\pi_{act}(e) = \texttt{collect}$ and to the objects $\pi_{obj}(e) = \{f_1, h_1, w_1, w_2\}$, where $ot(f_1) = \mathit{Frame}$

Definition 2. *Given a set of object types Σ, and a set of activities $\mathcal{A}$, an* object-centric Petri net *(OCPN) is defined as a tuple $N = (P, T, F, pt, F_{var}, \ell)$, where:*

1. *P and T are finite sets of places and transitions such that $P \cap T = \emptyset$;*
2. *$F \subseteq (P \times T) \cup (T \times P)$ is the flow relation between places and transitions;*
3. *$pt\colon P \to \Sigma$ maps every place to an object type;*
4. *$F_{var} \subseteq F$ is the set of variable flow relations;*
5. *$\ell\colon T \to \mathcal{A} \cup \{\tau\}$ is the transition labelling where τ marks an invisible activity;*

The pre-places of a transition t are all places with ingoing flow relations $\bullet t = \{p \in P | (p, t) \in F\}$, while the post-places of t are all those connected with outgoing flow relations $t\bullet = \{p \in P | (t, p) \in F\}$. For a transition t, the function $tpl(t)$ returns the types of all its corresponding places $p \in \bullet t \cup t\bullet$. $tpl_{var}(t)$ returns all place types connected via variable flow, and $tpl_{nv}(t)$ all place types connected via non-variable flow. An OCPN is considered well-formed if all flow relations of one transition agree in the variability for the place types, i.e., $\forall t \in T : tpl_{var}(t) \cap tpl_{nv}(t) = \emptyset$.

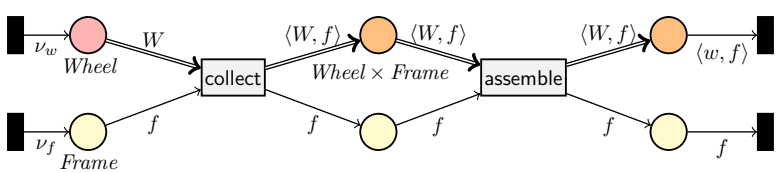
We pose the following specific assumptions about the structure of an OCPN. The contemporary discovery algorithms for OCPN utilize the Inductive Miner algorithm [8], which results in sound workflow nets for each object type. As a result, all places are bound to at most one object identity by design. Therefore, we interpret the marking M of an OCPN as a set of objects in a marking such that $M\colon Q \to \{0,1\}$. Furthermore, each object type has one initial place and one final place.

The set of binding executions is $B = \{(t,b) \in T \times (\Sigma \to \mathcal{P}(\mathcal{O})) \mid dom(b) = tpl(t) \wedge \forall ot \in tpl_{nv}(t) : |b(ot)| = 1\}$. It contains pairs of a transition t and a binding b. A binding b is a function that maps an object type $ot \in \Sigma$ to a set of objects. We denote by $codom(b)$ the codomain of b, i.e., $codom(b) = \bigcup_{\sigma \in \Sigma} b(\sigma)$. This binding execution consumes a multiset of object tokens from the transition's pre-places $cons(t,b) = [(p,o) \in Q \mid p \in \bullet t \wedge o \in b(pt(p))]$. Similarly, it produces object tokens $prod(t,b) = [(p,o) \in Q \mid p \in t \wedge \bullet\, o \in b(pt(p))]$ This binding is enabled in a marking M_i and can be executed if $cons(t,b) \leq M_i$. The resulting marking is $M_j = M_i - cons(t,b) + prod(t,b)$. The firing is denoted as $M_i \xrightarrow{(t,b)} M_j$. $B \subseteq T \times \mathcal{O}$ is the set of all possible bindings, such that a binding (t,b) for a transition t considers only objects $b \subseteq \mathcal{O}$ that belong to the types of the respective input places and are in line with the variable and non-variable arcs.

An accepting OCPN combines an OCPN N with an initial marking $M_{init} \in \mathcal{B}(Q)$ and a final marking $M_{final} \in \mathcal{B}(Q)$. An accepted run is a sequence of binding executions from the initial to the final marking $M_{init} \xrightarrow{*} M_{final}$

2.2 Object-centric Petri nets with identifiers

Example 2 (Simple OPID for vehicle manufacturing). The OPID combines the object types *Wheel* (red), *Frame* (yellow), and relations for both (orange places).



We summarize the definitions from [5]. Every object type $\sigma \in \Sigma$ is assumed to have a domain $dom(\sigma) \subseteq \mathcal{O}$, given by all objects in $\mathcal{O}$ of type σ. List types with a base type in σ are denoted as $[\sigma]$. Each Petri net place has a *color* that is a cartesian product of data types from Σ; precisely, the set of colors $\mathcal{C}$ is the set of all $\sigma_1 \times \cdots \times \sigma_m$ such that $m \geq 1$ and $\sigma_i \in \Sigma$ for all $1 \leq i \leq m$. We fix a set of Σ-typed variables $\mathcal{X} = X \uplus X_{list} \uplus \Upsilon$ as the disjoint union of a set X of "normal" variables that refer to single objects, denoted by lower-case letters like x, with a type $\text{type}(x) \in \Sigma$, a set X_{list} of list variables that refer to a list of objects of the same type, denoted by upper case letters like U, with a type $\text{type}(U) = [\sigma]$ for some $\sigma \in \Sigma$, and a set Υ of variables referring to fresh objects, denoted as ν, with $\text{type}(\nu) \in \Sigma$.

In OPIDs, tokens are object tuples associated with a color. To define relationships between objects in consumed and produced tokens when firing a transition, arc *inscriptions* are defined.

Definition 3. *An* inscription *is a tuple* $\boldsymbol{x} = \langle x_1, \ldots, x_m \rangle$ *such that* $m \geq 1$ *and* $x_i \in \mathcal{X}$ *for all* i*, but at most one* $x_i \in X_{list}$*, for* $1 \leq i \leq m$*. We call* $\boldsymbol{x}$ *a* template inscription *if* $x_i \in X_{list}$ *for some* i*, and a* simple inscription *otherwise.*

Template inscriptions are used to capture an arbitrary number of tokens of the same color; they are used to label the correspondent of variable arcs in OCPNs. The color of an inscription $\iota = \langle x_1, \ldots, x_m \rangle$ is given by $\mathrm{color}(\iota) = \langle \sigma_1, \ldots, \sigma_m \rangle$ where $\sigma_i = \mathrm{type}(x_i)$ if $x_i \in X \cup \Upsilon$, and $\sigma_i = \sigma'$ if x_i is a list variable of type $[\sigma']$. We set $\mathrm{vars}(\iota) = \{x_1, \ldots, x_m\}$. The set of all inscriptions is denoted Ω.

Definition 4. *An* object-centric Petri net with identifiers *(OPID) is defined as a tuple* $N = (\Sigma, P, T, F_{in}, F_{out}, \mathrm{color}, \ell)$*, where:*

1. P *and* T *are finite sets of places and transitions such that* $P \cap T = \emptyset$*;*
2. $\mathrm{color}\colon P \to \mathcal{C}$ *maps every place to a color;*
3. $\ell\colon T \to \mathcal{A} \cup \{\tau\}$ *is the transition labelling where* τ *marks an invisible activity,*
4. $F_{in}\colon P \times T \to \Omega$ *is a partial function called* input flow*, such that* $\mathrm{color}(F_{in}(p,t)) = \mathrm{color}(p)$ *for every* $(p,t) \in dom(F_{in})$*;*
5. $F_{out}\colon T \times P \to \Omega$ *is a partial function called* output flow*, such that* $\mathrm{color}(F_{out}(t,p)) = \mathrm{color}(p)$ *for every* $(t,p) \in dom(F_{out})$*;*

We set $\mathrm{vars}_{in}(t) = \bigcup_{p \in P} \mathrm{vars}(F_{in}(p,t))$*, and* $\mathrm{vars}_{out}(t) = \bigcup_{p \in P} \mathrm{vars}(F_{out}(t,p))$*, and require that* $\mathrm{vars}_{in}(t) \cap \Upsilon = \emptyset$ *and* $\mathrm{vars}_{out}(t) \subseteq \mathrm{vars}_{in}(t) \cup \Upsilon$*, for all* $t \in T$*.*

If $F_{in}(p,t)$ is defined, it is called a *variable* flow if $F_{in}(p,t)$ is a template inscription, and *non-variable* flow otherwise; and similar for output flows. Variable flows play the role of variable arcs in OCPNs [14]; they can carry multiple tokens at once. The common notations $\bullet t = \{p \mid (p,t) \in dom(F_{in})\}$ and $t\bullet = \{p \mid (t,p) \in dom(F_{out})\}$ are used as well.

Semantics. Given the set of objects $\mathcal{O}$, the set of *tokens* $\boldsymbol{\mathcal{O}}$ is the set of object tuples $\boldsymbol{\mathcal{O}} = \{\mathcal{O}^m \mid m \geq 1\}$. The *color* of a token $\omega \in \boldsymbol{\mathcal{O}}$ of the form $\omega = \langle o_1, \ldots, o_m \rangle$ is denoted $\mathrm{color}(\omega) = \langle \mathrm{type}(o_1), \ldots, \mathrm{type}(o_m) \rangle$. A *marking* of an OPID $N = \langle \Sigma, P, T, F_{in}, F_{out}, \mathrm{color}, \ell \rangle$ is a function $M\colon P \to 2^{\boldsymbol{\mathcal{O}}}$, such that for all $p \in P$ and $\langle o_1, \ldots o_m \rangle \in M(p)$, it holds that $\mathrm{color}(\langle o_1, \ldots o_m \rangle) = \mathrm{color}(p)$. Let $Lists(\mathcal{O})$ denote the set of objects lists of the form $[o_1, \ldots, o_k]$ with $o_1, \ldots, o_k \in \mathcal{O}$ such that all o_i have the same type; the type of such a list is then $[\mathrm{type}(o_1)]$. Next, *bindings* are defined to fix which objects are involved in a transition firing.

Definition 5. *A* binding *for a transition* t *and a marking* M *is a type-preserving function* $\beta\colon \mathrm{vars}_{in}(t) \cup \mathrm{vars}_{out}(t) \to \mathcal{O} \cup Lists(\mathcal{O})$*. To ensure the freshness of created values,* β *is required to be injective on* $\Upsilon \cap \mathrm{vars}_{out}(t)$*, and* $\beta(\nu)$ *must not occur in* M *for all* $\nu \in \Upsilon \cap \mathrm{vars}_{out}(t)$*.*

We denote by $codom(\beta) \subseteq \mathcal{O}$ the set of all objects occurring in tuples returned by β. Bindings are extended to inscriptions, denoted $\boldsymbol{\beta}$, to fix which tokens participate in a transition firing. For an inscription $\iota = \langle x_1, \ldots, x_m \rangle$, let $o_i = \beta(x_i)$ for all $1 \leq i \leq m$. Then $\boldsymbol{\beta}(\iota)$ is the set of object tuples defined as follows: if ι is a simple inscription, then $\boldsymbol{\beta}(\iota) = \{\langle o_1, \ldots, o_m \rangle\}$. Otherwise, there must be one x_i, $1 \leq i \leq n$, such that $x_i \in X_{list}$, and consequently o_i must be a list, say $o_i = [u_1, \ldots, u_k]$ for some $u_1, \ldots, u_k$. Then $\boldsymbol{\beta}(\iota) = \{\langle o_1, \ldots, o_{i-1}, u_1, o_{i+1}, \ldots, o_m \rangle, \ldots, \langle o_1, \ldots, o_{i-1}, u_k, o_{i+1}, \ldots, o_m \rangle\}$. $\mathcal{B}$ denotes the set of all bindings. Finally, a transition with a binding β is enabled if all object tuples pointed by $\boldsymbol{\beta}$ occur in the current marking:

Definition 6. *A transition $t \in T$ and a binding β for marking M are* enabled *in M if $\boldsymbol{\beta}(F_{in}(p,t)) \subseteq M(p)$ for all $p \in \bullet t$.*

Definition 7. *Let transition t be enabled in marking M with binding β. The* firing *of t yields the new marking M' given by $M'(p) = M(p) \setminus \boldsymbol{\beta}(F_{in}(p,t))$ for all $p \in \bullet t \setminus t\bullet$, and $M'(p) = M(p) \cup \boldsymbol{\beta}(F_{out}(p,t))$ for all $p \in t\bullet \setminus \bullet t$, and $M'(p) = M(p) \setminus \boldsymbol{\beta}(F_{in}(p,t)) \cup \boldsymbol{\beta}(F_{out}(p,t))$ for all $p \in \bullet t \cap t\bullet$.*[1]

We write $M \xrightarrow{t,\beta} M'$ to denote that t is enabled for binding β in M, and its firing yields M'. An *accepting* OPID N is an OPID together with a set of initial markings M_{init} and a set of final markings M_{final}. A run is accepted if it starts in the initial and ends in the final marking $M_{init} \xrightarrow{*} M_{final}$.

3 Equivalent Object-centric Petri Nets with Identifiers

This section defines the mapping $\mathbb{T}_1$ from an OCPN N to an OPID. Second, we show that the replay of both nets N and $\mathbb{T}_1(N)$ on a given event log are equivalent, i.e., they accept the same executions and reject the same executions.

Mapping to OPID. In this paper, we assume that for any given OCPN $N = (P, T, F, pt, F_{var}, \ell)$ there are fixed sets of places $P_{\blacktriangleright}, P_{\blacksquare} \subseteq P$ called *play* and *stop places*, respectively, such that for each $\sigma \in \Sigma$ there is exactly one place $p \in P_{\blacktriangleright}$ and $p' \in P_{\blacksquare}$ such that $pt(p) = pt(p') = \sigma$. These are the places that are supposed to be populated by initial and final markings, respectively. Markings M_{init} and M_{final} are *suitable* initial and final markings for an OCPN with play and stop places $P_{\blacktriangleright}, P_{\blacksquare}$ if they put tokens only in those places, i.e., for all $o \in \mathcal{O}$, $M_{init}(p, o) = 0$ for all $p \notin P_{\blacktriangleright}$ and $M_{init}(p', o) = 0$ for all $p' \notin P_{\blacksquare}$.

Note that a given set of objects uniquely determines suitable initial and final markings for N: precisely, for a set of objects $O \subseteq \mathcal{O}$, let M^O_{init} (resp. M^O_{final}) be the marking such that for each $\sigma \in \Sigma$ and and each $o \in O_\sigma$, $M^O_{init}(p_\sigma, o) = 1$ (resp. $M^O_{init}(p_\sigma, o) = 1$) where p_σ is the unique place in $P_{\blacktriangleright}$ (resp. $P_{\blacksquare}$) of type σ.

Our mapping $\mathbb{T}_1$ maps an OCPN N with start and stop places to an OPID $\mathbb{T}_1(N)$ which keeps all places and transitions of N. The flow relations are adjusted

[1] The case for $p \in \bullet t \cap t\bullet$ was omitted in [5], which is insufficient for cyclic nets.

and extended with arc inscriptions. Moreover, $\mathbb{T}_1(N)$ has additional transitions for creating and removing objects. In this way, the OPID can, on-demand, populate the places in $P_{\blacktriangleright}$ with objects in an initial marking for N and consume objects from places in $P_{\blacksquare}$ once a final marking for N is reached. The initial and final markings for $\mathbb{T}_1(N)$ are always empty.

Precisely, the OPID has an emitting transition for every object type, i.e., we set $T_e := \{t_{e\sigma} \mid \sigma \in \Sigma\}$; and a consuming transition for every object type i.e., we set $T_c := \{t_{c\sigma} \mid \sigma \in \Sigma\}$. With its emitting transitions T_e, an OPID allows for creating arbitrary new objects. However, under the replay of an OCEL L, this set of objects is fixed and denoted as $\mathcal{O}_L$. Hence, when replaying the log L, the inscription variables $\nu_i \in F_{out}(T_e)$ will be bound exclusively to objects in $\mathcal{O}_L$. To be able to define suitable inscriptions, we assume that for every object type $\sigma \in \Sigma$ there is a normal variable $x_\sigma \in X$, a list variable $X_\sigma \in X_{list}$, and a fresh object variable $\nu_\sigma \in \Upsilon$. We then set:

$$F_{out}\colon (T \cup T_e) \times P \to \Omega \qquad F_{out}(t,p) = \begin{cases} \langle x_{pt(p)} \rangle & \text{if } (t,p) \in F \setminus F_{var} \\ \langle X_{pt(p)} \rangle & \text{if } (t,p) \in F_{var} \\ \langle \nu_{pt(p)} \rangle & \text{if } p \in P_{\blacktriangleright} \wedge t \in T_e \end{cases}$$

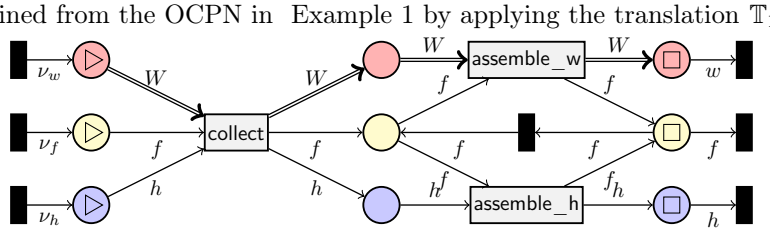
$$F_{in}\colon P \times (T \cup T_c) \to \Omega \qquad F_{in}(p,t) = \begin{cases} \langle x_{pt(p)} \rangle & \text{if } (p,t) \in F \setminus F_{var} \\ \langle X_{pt(p)} \rangle & \text{if } (p,t) \in F_{var} \\ \langle x_{pt(p)} \rangle, & \text{if } p \in P_{\blacksquare} \wedge t \in T_c \end{cases}$$

The equivalent OPID is given by $\mathbb{T}_1(N) = (\Sigma_1, P_1, T_1, F_{in}, F_{out}, \mathit{color}, \ell_1)$ where $\Sigma_1 = \Sigma$, $P_1 = P$, $T_1 = T \cup T_e \cup T_c$, $\mathit{color}(p) = \langle pt(p) \rangle$ for all $p \in P$, and

$$\ell_1\colon T \to \mathcal{A} \qquad \ell_1(t) = \begin{cases} \tau, & \text{if } t \in T_e \cup T_d \\ \ell(t), & \text{otherwise} \end{cases}$$

$\mathbb{T}_1(N)$ is interpreted as an accepting OPID where the initial and final markings are defined as the empty marking $M_\emptyset$, which requires all places to be empty.

Example 3 (Equivalent OPID for vehicle manufacturing). The following OPID is obtained from the OCPN in Example 1 by applying the translation $\mathbb{T}_1$.



The example OPID can replay the logs L_1 as well as L_2 as it has equivalent semantics to Example 1 in combining objects without additional constraints.

Proof of Equivalence. To prove the language equivalence of N and $\mathbb{T}_1(N)$ in general, we first define some auxiliary notions.

Given a marking $M\colon Q \to \{0,1\}$ of the OCPN N, we define the corresponding marking $\mathbb{T}_1(M)$ of the OPID $\mathbb{T}_1(N)$ as $\mathbb{T}_1(M)(p) := \{\langle o\rangle \mid o \in \mathcal{O} \wedge M(p,o) > 0\}$.

Given a binding b for an OCPN N, we define a corresponding binding $\beta := \mathbb{T}_1(b)$ for the OPID $\mathbb{T}_1(N)$ as follows: for all $\sigma \in dom(b)$, we set $\beta(v_\sigma) = b(\sigma)$ if $|b(\sigma)| = 1$, and $\beta(V_\sigma) = [b(\sigma)]$ otherwise, where $[b(\sigma)]$ denotes a list containing the elements of the set $b(\sigma)$ in an arbitrary order.

Moreover, we call a sequence of transition firings $M \xrightarrow{*} M'$ in $\mathbb{T}_1(N)$ an *emitting sequence* if it uses only emitting transitions in T_e and a *collapsing sequence* if it uses only collapsing transitions in T_c.

We can then show the following equivalence statement:

Theorem 1. *Let O be a set of objects, N be an accepting OCPN, $N' = \mathbb{T}_1(N)$, and $M_\emptyset$ denote the empty marking of N'.*

There exists a run of N:

$$M_0 \xrightarrow{(t_1,b_1)} M_1 \xrightarrow{(t_2,b_2)} \cdots \xrightarrow{(t_n,b_n)} M_n \tag{1}$$

with $M_0{=}M^O_{init}$ and $M_n{=}M^O_{final}$ if and only if there is a run of N' over the set of objects O:

$$M_\emptyset \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset \tag{2}$$

such that $M'_i = \mathbb{T}_1(M_i)$ for all $0 \le i \le n$ and $\beta_i = \mathbb{T}_1(b_i)$ for all $1 \le i \le n$, such that $M_\emptyset \xrightarrow{*} M'_0$ is an emitting sequence, $M'_n \xrightarrow{*} M_\emptyset$ a collapsing sequence, and no emitting or collapsing transitions occur in between.

In the following, we outline the rational for proving Theorem 1, while the detailed proof is spelled out in Appendix A of the extended paper[2] .

4 Synchronization for Stable Many-to-One Relationships

In this section, we first define stable many-to-one relationships for an object-centric process execution. For a given OCEL and a replaying OCPN, we then extend the mapping $\mathbb{T}_1$ from Section 3 to implement the expected synchronization for the expected stable many-to-one relationship. Finally, we prove that any execution of the OCPN is accepted by the mapped OPID if it conforms to the stable many-to-one relationships, and vice versa.

In the following, let a pair (σ_m, σ_o) be an *object type relationship* if $\sigma_m \neq \sigma_o$. For the vehicle manufacturing example, a stable many-to-one relationship may exist for *(Wheel, Frame)*. An event log and analog an execution of an OCPN or OPID should conform to this stable relationship.

Given an OCEL $L = (E, \mathcal{O}, \pi_{act}, \pi_{obj}, \pi_{time})$, we define the function $lo\colon \mathcal{O} \times \Sigma \to \mathcal{O}$, which given an object and an object type, returns all co-occurring objects of this type, i.e., $lo(o,\sigma) = \bigcup_{e\in E}\{o' \mid o, o'\pi_{obj}(e_1) \wedge ot(o') = \sigma\}$. For OCPNs and OPIDs, lo is defined accordingly over a run of the net and bindings.

[2] Extended version avilable at: `tbd`

Definition 8. *An object type relationship* (σ_m, σ_o) *is a* stable many-to-one relationship *in an event log* L *if for all objects* $o_m, o_o \in \mathcal{O}$*, where* $ot(o_m) = \sigma_m$ *and* $ot(o_o) = \sigma_o$ *it holds that* $|lo(o_m, \sigma_o)| = 1$ *and* $lo(o_o, \sigma_m) \neq \emptyset$*.*

Anjo: Check if we need both definitions

Analog for OCPNs and OPIDs, (σ_m, σ_o) is a *stable many-to-one relationship* if the same holds for a run in the net.

For example, *(Wheel, Frame)* is a stable many-to-one relationship in the log L_1 from Section 1, but not in L_2. Note that the one-to-one relationship between handlebar and frame manifests as a bidirectional relationship where *(Handlebar, Frame)* and *(Handlebar, Frame)* are stable many-to-one relationships.

4.1 Mapping to synchronizing OPIDs

The goal of this mapping is to enforce a set of stable many-to-one relationships with the synchronization behavior of an OPID. To this end, for every many-to-one relationship (σ_m, σ_o), the required set of links $(m, o) \in \mathcal{O}^2$ of type (σ_m, σ_o) must be determined *before* the actual execution starts, and it must be stable during the execution. In general, the idea is to store all links of a relationship in a dedicated place, which is accessed by all transitions that operate on both object types. The transition's synchronization semantics ensure that only objects in a stable relationship are in a binding.

We assume a given OCPN $N = (P, T, F, pt, F_{var}, \ell)$, and a fixed set of object type relationships R, that are supposed to work as stable many-to-one relationships. The mapping $\mathbb{T}_R$ is defined on top of the mapping $\mathbb{T}_1$ in Section 3. Let $N_1 = \mathbb{T}_1(N) = (\Sigma_1, P_1, T_1, F_{in,1}, F_{out,1}, \text{color}_1, \ell_1)$. To define the synchronizing mapping $\mathbb{T}_R(N_1)$, we set $\text{color}_2 = \text{color}_1$, $F_{in,2} = F_{in,1}$, and $F_{out,2} = F_{out,1}$ and extend them in the following.

We create a new place for every pair of object types in R, and set $P_L := \{p_{(\sigma_m,\sigma_o)} \mid (\sigma_m, \sigma_o) \in R\}$, such that $\text{color}_2(p_{(\sigma_m,\sigma_o)}) = \langle \sigma_m, \sigma_o \rangle$.

All links ought to be instantiated before the emitting transitions $T_e \subset T_1$ place objects to the initial places $P_\blacktriangleright \subset P_1$. The link creation requires multiple additional transitions and places. A new emitting transition is added for every object type that occurs in R; to this end we set $T_i := \{t_{i,\sigma} \mid \exists \sigma' \in \Sigma : (\sigma, \sigma') \in R \vee (\sigma', \sigma) \in R\}$. After the creation of the objects, a second set of transitions will create the required links for every combination of object types with a stable many-to-one relationship: let $T_L := \{t_{\{\sigma_m,\sigma_o\}} \mid (\sigma_m, \sigma_o) \in R\}$ To allow for concurrently creating all required links, a place is added before and after each link creation transition for every object type. The set of places before the creation is $P_b := \{p_{\sigma,(\sigma_m,\sigma_o)} \mid (\sigma = \sigma_m \vee \sigma = \sigma_o) \wedge (\sigma_m, \sigma_o) \in R\}$ such that $\text{color}_2(p_{\sigma,(\sigma_m,\sigma_o)}) = \sigma$. Similarly, the set of places after link creation is $P_a := \{p_{(\sigma_m,\sigma_o),\sigma} \mid (\sigma = \sigma_m \vee \sigma = \sigma_o) \wedge (\sigma_m, \sigma_o) \in R\}$ such that $\text{color}_2(p_{\sigma,(\sigma_m,\sigma_o)}) = \sigma$.

The transitions T_i, T_L, T_e and places P_b and P_a are connected through flow relations in such a way that all links are created for the many-to-one relationships concurrently before they are placed on the initial places by T_e. Therefore, for any $t_{i,\sigma} \in T_i$, and $p \in P_b$ where $\text{color}_2(p) = \sigma$, the outflow is set as

$F'_{out}(t_{i,\sigma}, p) = \langle \nu_\sigma \rangle$. All former emitting transitions $t \in T_e$ will operate on existing objects produced by the transitions in T_i if the object type requires links. Hence, if $F_{out}(t_e, p) = \langle \nu_{\mathrm{color}_2(p)} \rangle$ and a $\sigma \in \Sigma$ exists such that $(\sigma, \mathrm{color}_2(p))$ or $(\mathrm{color}_2(p), \sigma)$ exist in R, we set $F'_{out}(t_e, p) = \langle x_{\mathrm{color}_2(p)} \rangle$. Every link creation transition $t_{\{\sigma_m,\sigma_o\}} \in T_L$ is connected with inflow to the according places from P_b and with outflow to the places in P_a. If the place $p_{\sigma_o,(\sigma_m,\sigma_o)} \in P_b$ is for the one-side of the relationship, then the inflow reads a single object $F'_{in}(p, t) = \langle x_{\mathrm{color}_2(p)} \rangle$. And if the place $p_{\sigma_m,(\sigma_m,\sigma_o)} \in P_b$ is for the many-side of the relationship, then the inflow reads a variable list of objects $F'_{in}(p, t) = \langle X_{\mathrm{color}_2(p)} \rangle$. Similarly, the outflow between a transition $t_{\{\sigma_m,\sigma_o\}} \in T_L$ and P_a is set as follows. If the place $p_{(\sigma_m,\sigma_o),\sigma_o} \in P_a$ is for the one-side of the relationship, then the outflow produces a single object $F'_{out}(t_{\{\sigma_m,\sigma_o\}}, p) = \langle x_{\mathrm{color}_2(p)} \rangle$. And if the place $p_{(\sigma_m,\sigma_o),\sigma_m} \in P_b$ is for the many-side of the relationship, the outflow produces a variable list of objects $F'_{out}(t_{\{\sigma_m,\sigma_o\}}, p) = \langle X_{\mathrm{color}_2(p)} \rangle$. Every link creation transition $t_{\{\sigma_m,\sigma_o\}} \in T_L$ has an outflow to the according link place $p_{(\sigma_m,\sigma_o)} \in P_L$ with the arc inscription $F'_{out}(t_{\{\sigma_m,\sigma_o\}}, p_{(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m}, x_{\sigma_o} \rangle$.

After all objects have been concurrently processed for their intended links respectively, they are placed on the places in P_a. To merge the concurrent processing, every $t_{e,\sigma} \in T_e$ has an inflow from all related $p \in P_a$ where $\mathrm{color}_2(p) = \sigma$ such that $F'_{in}(p, t_{e,\sigma}) = \langle x_\sigma \rangle$. After all transitions T_e emitted the objects to the places $P_\blacktriangleright$, the places in P_L are populated with a fixed set of links.

To enforce the synchronization for the transitions T when reading related objects from places P of the initial OCPN N, the transitions need to read and write the required links from the link places P_L. Therefore, for all places $p_{(\sigma_m,\sigma_o)} \in P_L$ and transitions $t \in T$ where there exist places $p_1, p_2 \in P$ such that $\mathrm{color}_2(p_1) = \sigma_m$ and $\mathrm{color}_2(p_2) = \sigma_o$ and there exists $F_{in}(p_1, t)$ and $F_{in}(p_2, t)$, then $F'_{in}(p_{(\sigma_m,\sigma_o)}, t) = F'_{out}(t, p_{(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m}, x_{\sigma_o} \rangle$. Note that we assume that the OCPN is constructed from sound workflow nets. Hence, every transition has an inflow for an object type if and only if it has an outflow for it.

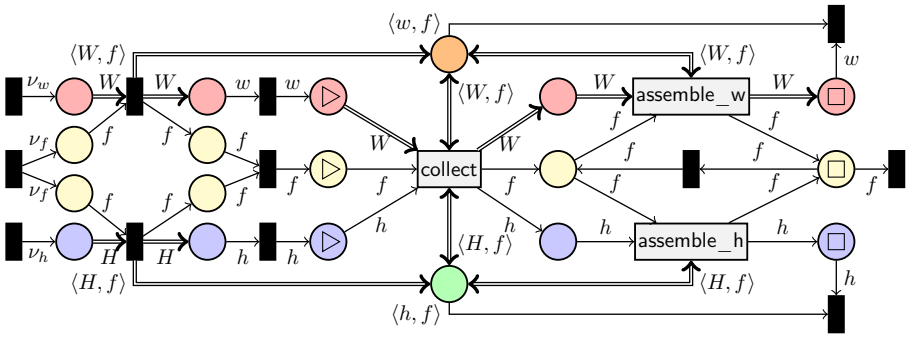
Finally, all objects are consumed by their according consuming transition $t_{c,\sigma} \in T_c$. We extend the inflow of the transition to consume an object together with all its many-to-one links. For all link places $p_{(\sigma_m,\sigma_o)} \in P_L$ where $\sigma_m = \sigma$, an arc consumes the links $F'_{in}(p_{(\sigma_m,\sigma_o)}, t_{c,\sigma_m}) = \langle X_{\sigma_m}, x_{\sigma_o} \rangle$.

The new places are $P_2 = P_1 \cup P_b \cup P_L \cup P_a$ and the new transitions are $T_2 = T_1 \cup T_i \cup T_L$. As a result, the mapping $\mathbb{T}_R$ results in the following OPID $\mathbb{T}_R(\mathbb{T}_1(N)) = (\Sigma_1, P_2, T_2, F_{in,2}, F_{out,2}, \mathrm{color}_2, \ell_2)$ where

$$\ell_2 \colon T \to \mathcal{A} \qquad\qquad \ell_2(t) = \begin{cases} \tau, & \text{if } t \in T_i \cup T_L \\ \ell_1(t), & \text{otherwise} \end{cases}$$

The OPID N_2 is again interpreted as an accepting OPID where the initial marking $M_{init,2}$ and final marking $M_{final,2}$ require all places to be empty.

Example 4 (Synchronizing OPID for vehicle manufacturing). The OPID is obtained from the OCPN in Example 1 by applying the translations $\mathbb{T}_1$ and $\mathbb{T}_R$ for the stable relationships *(Wheel, Frame)* (orange) and *(Handlebar, Frame)* (green).



The OPID can accept the run of log L_1 because there exists a run, where the links for wheels and frames can be set as $(w_1, f_1), (w_2, f_1), (w_3, f_2), (w_4, f_2)$ in the orange link place and tokens for handlebars and frames as $(h_1, f_1), (h_2, f_2)$ in the green link place. The transitions can bind the objects according to the links. However, the run in log L_2 will be rejected, because every wheel can only take part in one link token. The execution of $\langle \mathsf{collect}, \{f_3, h_3, w_5, w_6\}\rangle$ requires the links (w_5, f_3) and (w_6, f_3) but $\langle \mathsf{assemble_w}, \{f_4, w_6, w_7\}\rangle$ requires the link (w_6, f_4), which can not exist at the same time.

4.2 Proof of equivalence for executions with stable relationships

In Section 3, the equivalence of an OCPN N and its OPID representation $N_1 = \mathbb{T}_R(N)$ for replaying an event log is shown. The mapping $\mathbb{T}_R$ implements the stable relationships R into N_1. We prove the correct mapping by showing two things: First, any accepted run in N_2 is also accepted in N_1. Second, any accepted run in N_1 that is conform to the stable relationships R, is accepted by N_2.

Theorem 2. *Let $N_1 = \mathbb{T}_1(N) = (\Sigma_1, P_1, T_1, F_{in,1}, F_{out,1}, \mathrm{color}_1, \ell_1)$ be the equivalent OPID for the accepting OCPN N over the set of objects $\mathcal{O}$. Let $N_2 = \mathbb{T}_R(N_1) = (\Sigma_2, P_2, T_2, F_{in,2}, F_{out,2}, \mathrm{color}_2, \ell_2)$ for the set R of stable object type relationships, and $M_\emptyset$ denote the empty marking of N_1 and N_2.*
(1) For any accepted run of N_2:

$$M_\emptyset \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset$$

such that $M_\emptyset \xrightarrow{*} M'_0$ is an emitting and link creation sequence of only transitions in $t \in T_i \cup T_L \cup T_e$, $M'_n \xrightarrow{*} M_\emptyset$ is a collapsing sequence of only transitions $t \in T_c$, and no transitions $t \in T_i \cup T_L \cup T_e \cup T_c$ occur in between, there is an accepted run of N_1 of the form

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset \tag{3}$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ for all $0 \leq i \leq n$.
(2) For an accepted run of N_1 that respects the stable relationships R:

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset$$

there is an accepted run of N_2 of the form

$$M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset \tag{4}$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ *for all* $0 \leq i \leq n$.

Anjo: Added for prooving that a non-conform run is not accepted by the OPID.

(3) For an accepted run of N_1 that does not respect the stable relationships R:

$$M_\emptyset \xrightarrow{*} M_0 \xrightarrow{(t_1,\beta_1)} M_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M_n \xrightarrow{*} M_\emptyset$$

there is no accepted run of N_2 of the form

$$M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0 \xrightarrow{(t_1,\beta_1)} M'_1 \xrightarrow{(t_2,\beta_2)} \cdots \xrightarrow{(t_n,\beta_n)} M'_n \xrightarrow{*} M_\emptyset \tag{5}$$

such that $\forall p \in P_1 \colon M'_i(p) = M_i(p)$ *for all* $0 \leq i \leq n$.

In the following, we outline the rational for proving Theorem 2, while the detailed proof is spelled out in Appendix B of the extended paper[3] .

By combining Theorem 1 and Theorem 2, we prove that for a given accepting OCPN N, there is an equivalent OPID $\mathbb{T}_1(N)$ that can be extended with stable relationships as $\mathbb{T}_R(\mathbb{T}_1(N))$ such that for any accepted run of N that satisfies the stable relationships R there is an accepted run in $\mathbb{T}_R(\mathbb{T}_1(N))$. By counter-examples it is shown that an OCPN N wrongly accepts executions that violate stable many-to-one relationships, while $\mathbb{T}_R(\mathbb{T}_1(N))$ is able to reject such executions.

Anjo: If we have the proof for Item (3), we don't need counter-examples anymore

5 Discussion and Related Work

Our paper provides the formal guarantee that OCPNs are well-suited for replaying the behavior of event logs, which satisfy the expected stable many-to-one relationships. Such relationships can be discovered from OCELs, as described for the tool TOTeM [9]. In existing benchmark logs[4], TOTeM discovers many-to-one and many-to-many relationships that hold for the whole log. Even in a world use case, Xiu and Li [19] identify stable many-to-one relationships and observe their violation. It needs to be noted, that our definition of stable many-to-one relationships only considers existential relations of the type `(1..*,1..1)`. Non-existential relationships of the type `(0..*,0..1)` should be considered in future investigations as well.

Anjo: We could simply run TOTeM for these logs. Should we visualize it somehow?

In the literature, object relationships are typically assumed to be stable. In the widely adopted OCEL 2.0 standard [2], they are recorded without a notion of time assuming global stability. For discovered models, such as OCPNs [14], class diagrams [9], and even simulation models [7], this results in the same notion of stable relationships. However, Goossens et al. [6] point out that from a database perspective links between objects might change over time, and the

[3] Extended version avilable at: `tbd`

[4] `https://www.ocel-standard.org`

proposed format of object-centric event data implies that links may be related to time [11]. The prospect of changing relationships will require the discovery of more sophisticated formalisms that enforce synchronization.

Still, the current discovery of object-centric Petri nets is bound to stable relationships. The initial formalism of OCPN [14] was recently extended by van Detten et al. [16], who propose the discovery of object-centric process trees. Identifier sound OCPNs can be generated from such process trees, yet, without synchronization semantics. On the other hand, van Detten et al. [17] define the discovery of silent objects implementing synchronization for arbitrary sets of objects. In the future, these silent objects may serve as reification objects to also extend our investigation to many-to-many relationships.

As OPIDs can emit novel objects, they can not only serve as trace recognizers but also as trace generators. Therefore, our proposed mapping can serve as a missing link between existing works on object-centric process discovery and the formal analysis and execution of such processes. For instance, van der Werf et al. [15] define correctness notions for Petri nets with identifiers, which can now be applied to OCPNs through our mapping. Finally, OPIDs can also serve as a goal formalism for object-centric process modeling approaches, enabling the comparison of model semantics and discovery results.

Object-centric conformance checking has mainly been studied under the identified assumptions of stable many-to-one relationships. The conformance metrics of fitness and precision have been introduced to quantify the appropriateness of an OCPN in regard to replaying a log [1]. However, the existing definitions only concern event orderings for individual objects but omit object relations. As discussed, this is just fine as long as stable relationships are not violated. For violating executions, OPIDs are defined to compute conformance alignments that include a penalty for violated relationships [5]. In the future, conformance metrics should respect synchronization behavior as defined in OPIDs. Other process models have been considered for object-centric conformance checking as well. Park et al. use object-centric directly follows graphs [10], which are more ambiguous in their synchronization behavior than OCPN. On the other hand, Xiu and Li define conformance checking for the declarative approach of object-centric behavioral constraints [18]with a consideration of object relationships.

6 Conclusion

This contribution explicates the typical assumption on object-centric process executions that object relations are stable. As a result, discovered OCPNs are too ambiguous to identify executions that violate expected stable many-to-one relations. Our mapping to non-ambiguous OPIDs implements the expected synchronization of objects for stable relationships. Executions that conform to these stable relationships can only bind related objects. Such executions are accepted by the OPID, serving as the formal guarantee that OCPNs are indeed "just fine" to replay these specific executions non-ambiguously.

In the future, the synchronization of many-to-many relations in event logs remains to be studied in detail. Furthermore, our mapping can serve as a promising link between object-centric process discovery and formal analysis as well as model-driven process executions.

References

1. Adams, J.N., van der Aalst, W.M.P.: Precision and Fitness in Object-Centric Process Mining. In: 2021 3rd International Conference on Process Mining (ICPM) (Oct 2021). https://doi.org/10.1109/ICPM53251.2021.9576886
2. Berti, A., et al.: OCEL (Object-Centric Event Log) 2.0 Specification (Mar 2024). https://doi.org/10.48550/arXiv.2403.01975
3. Carmona, J., Van Dongen, B., Solti, A., Weidlich, M.: Conformance Checking: Relating Processes and Models. Springer International Publishing (2018). https://doi.org/10.1007/978-3-319-99414-7
4. Ghahfarokhi, A.F., et al.: OCEL: A Standard for Object-Centric Event Logs. In: New Trends in Database and Information Systems. Springer International Publishing (2021). https://doi.org/10.1007/978-3-030-85082-1_16
5. Gianola, A., Montali, M., Winkler, S.: Object-Centric Conformance Alignments with Synchronization. In: Advanced Information Systems Engineering. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-61057-8_1
6. Goossens, A., De Smedt, J., Vanthienen, J.: Object-Centric Event Logs: Characteristics, Comparative Analysis and Road Map. In: Business Process Management Forum. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-70418-5_3
7. Knopp, B., Pourbafrani, M., van der Aalst, W.M.P.: Discovering Object-Centric Process Simulation Models. In: 2023 5th International Conference on Process Mining (ICPM) (Oct 2023). https://doi.org/10.1109/ICPM60904.2023.10271944
8. Leemans, S.J.J., Fahland, D., van der Aalst, W.M.P.: Scalable process discovery and conformance checking. Software & Systems Modeling **17**(2), 599–631 (May 2018). https://doi.org/10.1007/s10270-016-0545-x
9. Liss, L., Adams, J.N., van der Aalst, W.M.P.: TOTeM: Temporal Object Type Model for Object-Centric Process Mining. In: Business Process Management Forum. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-70418-5_7
10. Park, G., Adams, J.N., van der Aalst, W.M.P.: Conformance Checking and Performance Analysis Using Object-Centric Directly-Follows Graphs. In: Business Process Management Forum. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-70418-5_11
11. Swevels, A., Fahland, D., Montali, M.: Implementing Object-Centric Event Data Models in Event Knowledge Graphs. In: Process Mining Workshops. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-56107-8_33
12. van der Aalst, W.M.P.: Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data. In: Software Engineering and Formal Methods. Springer International Publishing (2019). https://doi.org/10.1007/978-3-030-30446-1_1
13. van der Aalst, W.M.P., et al.: Process Mining Manifesto. In: Business Process Management Workshops. Springer, Berlin, Heidelberg (2012). https://doi.org/10.1007/978-3-642-28108-2_19

14. van der Aalst, W.M.P., Berti, A.: Discovering Object-centric Petri Nets. Fundamenta Informaticae **175**(1-4) (Jan 2020). https://doi.org/10.3233/FI-2020-1946
15. van der Werf, J.M.E.M., Rivkin, A., Montali, M., Polyvyanyy, A.: Correctness Notions for Petri Nets with Identifiers. Fundamenta Informaticae **190**(2-4) (Jan 2024). https://doi.org/10.3233/FI-242169
16. Van Detten, J.N., Schumacher, P., Leemans, S.J.J.: Discovering Compact, Live and Identifier-Sound Object-Centric Process Models. In: 2024 6th International Conference on Process Mining (ICPM) (Oct 2024). https://doi.org/10.1109/ICPM63005.2024.10680659
17. van Detten, J.N., Schumacher, P., Leemans, S.J.J.: Object Synchronizations and Specializations with Silent Objects in Object-Centric Petri Nets. In: Business Process Management. Springer Nature Switzerland (2024). https://doi.org/10.1007/978-3-031-70396-6_4
18. Xiu, B., Li, G.: Diagnosing Conformance Between Object-Centric Event Logs and Models. IEEE Access **11** (2023). https://doi.org/10.1109/ACCESS.2023.3322366
19. Xiu, B., Li, G.: Quantifying Conformance Between Object-Centric Event Logs and Models. IEEE Access **12** (2023). https://doi.org/10.1109/ACCESS.2024.3421936

A Proof for Theorem 1

Proof. (1) First, by definition of T_e in N', there is an emitting sequence $M_\emptyset \xrightarrow{*} M'_0$ since $M'_0 = \mathbb{T}_1(M_0)$ has tokens only in places that are initial in N. Moreover, by definition of T_c there is a collapsing sequence $M'_n \xrightarrow{*} M_\emptyset$ because $M'_n = \mathbb{T}_1(M_n)$ has tokens only in places that are final in N.

Next, we show the following ($\star$): for all $1 \leq i \leq n$, a transition firing $M_{i-1} \xrightarrow{(t_i,b_i)} M_i$ implies $M'_{i-1} \xrightarrow{(t_i,\beta_i)} M'_i$. Note first that the binding $\beta_i = \mathbb{T}_1(b_i)$ is type-preserving since this also holds for b_i; and it is well-defined because transitions in an OCPN have at most one input place for each sort, so the transition t_i of the OPID $\mathbb{T}_1(N)$ cannot have two input flows with the same inscription. All inscriptions in $\mathbb{T}_1(N)$ are singleton tuples, so the extension $\boldsymbol{\beta}_i$ of β_i to inscriptions is given by $\boldsymbol{\beta}_i(\langle x_\sigma \rangle) = \{\langle \beta_i(x_\sigma) \rangle\}$ for variables $x_\sigma \in \Upsilon \cup X$, and by $\boldsymbol{\beta}_i(\langle X_\sigma \rangle) = \{\langle a \rangle \mid a \text{ occurs in } \beta_i(X_\sigma)\}$ for all $X_\sigma \in X_{list}$. In both cases, we have $\boldsymbol{\beta}_i(x_\sigma) = \boldsymbol{\beta}_i(X_\sigma) = \{\langle a \rangle \mid a \in b_i(\sigma)\}$.

Now, as t_i is enabled with b_i in marking M_{i-1} of N, $cons(t_i, b_i) \leq M_{i-1}$ holds for $cons(t_i, b_i) = \{(p, o) \mid p \in \bullet t_i \text{ and } o \in b_i(pt(p))\}$. We have $\boldsymbol{\beta}_i(F_{in}(p, t_i)) = \{\langle o \rangle \mid o \in b_i(pt(p))\}$, and therefore $\boldsymbol{\beta}_i(F_{in}(p, t_i)) \subseteq M'_{i-1}(p)$ for all $p \in \bullet t_i$, so that t_i is enabled in marking $\mathbb{T}_1(M_{i-1})$ of N'. The resulting marking in N is $M_i = (M_{i-1} \setminus cons(t_i, b_i)) \cup prod(t, b)$, for $prod(t, b) = \{(p, o) \mid p \in t\bullet \text{ and } o \in b(pt(p))\}$. The equivalent marking in N' is given by $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p, t_i))$ for all $p \in \bullet t_i \setminus t_i\bullet$, $M'_i(p) = M'_{i-1}(p) \cup \boldsymbol{\beta}_i(F_{out}(p, t_i))$ for all $p \in t\bullet_i \setminus \bullet t_i$, and $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p, t_i)) \cup \boldsymbol{\beta}_i(F_{out}(p, t_i))$ for all $p \in \bullet t_i \cap t\bullet_i$. We have $\boldsymbol{\beta}_i(F_{in}(p, t_i)) = \boldsymbol{\beta}_i(F_{out}(p, t_i)) = \{\langle o \rangle \mid o \in b_i(pt(p))\}$, so $M'_i = \mathbb{T}_1(M_i)$. Thus ($\star$) holds.

A simple inductive argument using ($\star$), in addition to the observations about emitting and collapsing sequences, shows shows that the run (2) is possible.

(2) First, since $M_\emptyset \xrightarrow{*} M'_0$ is an emitting sequence from the initial marking, M'_0 has tokens only in places that are initial in N. So there is a marking M_0 of N that is a valid initial marking of N and satisfies $M'_0 = \mathbb{T}_1(M_0)$.
Next, we show the following ($\star$): for all $1 \leq i \leq n$, given M_{i-1} which satisfies $M'_{i-1} = \mathbb{T}_1(M_{i-1})$, there are a marking M_i and a binding b_i that satisfy $M_{i-1} \xrightarrow{(t_i,b_i)} M_i$, $M'_i = \mathbb{T}_1(M_i)$, and $\beta_i = \mathbb{T}_1(b_i)$. Since $\mathbb{T}_1$ is injective on markings and bindings, M_i and b_i can actually be obtained from M'_i and β_i by applying the inverse of $\mathbb{T}_1$. As t_i is enabled in M'_{i-1}, it holds that $\boldsymbol{\beta}_i(F_{in}(p,t_i)) \subseteq M'_{i-1}(p)$ for all $p \in \bullet t_i$. Since $\boldsymbol{\beta}_i(F_{in}(p,t_i)) = \{\langle o\rangle \mid o \in b_i(pt(p))\}$ (see Item (1)), $cons(t_i,b_i) \leq M_{i-1}$ holds, so t_i is enabled in M_{i-1}. The resulting marking M'_i in N' satisfies $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p,t_i))$ for all $p \in \bullet t_i \setminus t_i\bullet$, $M'_i(p) = M'_{i-1}(p) \cup \boldsymbol{\beta}_i(F_{out}(p,t_i))$ for all $p \in t\bullet_i \setminus \bullet t_i$, and $M'_i(p) = M'_{i-1}(p) \setminus \boldsymbol{\beta}_i(F_{in}(p,t_i)) \cup \boldsymbol{\beta}_i(F_{out}(p,t_i))$ for all $p \in \bullet t_i \cap t\bullet_i$. The resulting marking M_i in N satisfies $M_i = (M_{i-1} \setminus cons(t_i,b_i)) \cup prod(t,b)$. Since $\boldsymbol{\beta}_i(F_{in}(p,t_i)) = \boldsymbol{\beta}_i(F_{out}(p,t_i)) = \{\langle o\rangle \mid o \in b_i(pt(p))\}$, it holds that $M'_i = \mathbb{T}_1(M_i)$. Thus ($\star$) holds and an inductive argument shows that the run (1) is possible.
Finally, since $M'_n \xrightarrow{*} M_\emptyset$ is a collapsing sequence to an empty marking, $M'_n = \mathbb{T}_1(M_n)$ has tokens only in places that are final in N, so M_{final} is a suitable final marking for N. □

B Proof for Theorem 2

Proof. (1) $M_\emptyset \xrightarrow{*} M'_0$ in N_2 produces a set of objects through T_i, creates links through T_L, and the transitions $t \in T_e$ populate the play places $p \in P_\blacktriangleright$. In M'_0, only the places $P_\blacktriangleright$ and P_L hold tokens, i.e. $M'_0(p) = \emptyset$ for all $p \in P_2 \setminus (P_\blacktriangleright \cup P_L)$, because no transition $t \in T \setminus (T_e \cup T_L$ can remove the remaining tokens from any place $p \in P_b \cup P_a$, as for all those places and transitions it holds that $F_{in}(p,t) = \bot$, and the execution would not be accepted for the final marking $M_\emptyset$.
For N_1, there exists a sequence $M_\emptyset \xrightarrow{*} M_0$ that can generate the same marking for the play places $p \in P_\blacktriangleright \subset P_1$ by firing the appropriate emitting transition for every object. Indeed, for all objects and places $p' \in P_\blacktriangleright \subset P_2$, where $o \in M'_0(p')$ and $\text{type}(o) = \sigma$, there exists an emitting transition $t \in T_e \subset T_1$ with an outflow to the according place $p \in P_\blacktriangleright \subset P_1$ with $\text{color}_2(p) = \sigma$ where $F_{out,1}(t,p) = \langle \nu_\sigma \rangle$. Therefore, the emitting transition can produce o onto p. Hence, given the marking M'_0 in N_2, N_1 can produce an equivalent marking M_0 for all places $p \in P_1$.
Next we show the following ($\star$): For all $1 \leq i \leq n$, a transition firing $M'_{i-1} \xrightarrow{(t_i,\beta'_i)} M'_i$ implies the existence of a firing $M_{i-1} \xrightarrow{(t_i,\beta_i)} M_i$ where for all $p \in P_1$ it holds that $M_{i-1}(p) = M'_{i-1}(p)$. For all transitions $t' \in T_1 \subset T_2$ of N_2 and $t \in T_1$ of N_1 and places $p' \in P_1 \subset P_2$ of N_2 and $p \in P_1$ of N_1, the inflow and outflow are the same for both nets $F_{in,1}(t,p) = F_{in,2}(t',p')$ and

$F_{out,1}(t,p) = F_{out,2}(t',p')$. Hence if the transition (t_i, β_i) is enabled in marking M'_{i-1} of N_2 then it is is also enabled in M_{i-1} of N_1, and the resulting markings satisfy $M_i(p) = M'_i(p)$ for all places $p \in P_1$.
In M'_n, only the places $P_\blacksquare$ and P_L hold tokens, so $M'_n(p) = \emptyset$ for all $p \in P_2 \setminus (P_\blacksquare \cup P_L)$. Otherwise, no transition $t \in T_c \subset T_2$ can remove the remaining tokens from any place p, as for all those places and transitions it holds that $F_{in}(p,t) = \bot$, and the execution would not be accepted for the final marking $M_\emptyset$. Consider the collapsing sequence $M'_n \xrightarrow{*} M_\emptyset$ in N_2 of only transitions $t' \in T_c$ and a marking M_n in N_1, where $M_n(p) = M'_n(p)$ for all places $p \in P_1$. There exists a collapsing sequence in N_1 consisting of only consuming transitions $t \in T_c$, resulting in $M_\emptyset$ as $F_{in,1}(p,t) = \langle x_{\mathrm{color}_2(p)} \rangle$ and $F_{out,1}(t,p') = \bot$ for any $p' \in P_1$.
A simple inductive argument using $(\star)$, and the observations about the emitting, link creating, and the collapsing sequences, show that the run (3) is possible.

(2) For the emitting sequence $M_\emptyset \xrightarrow{*} M_0$ in N_1 and the set of relations R, there is an emitting sequence $M_\emptyset \xrightarrow{*} M'_b \xrightarrow{*} M'_a \xrightarrow{*} M'_0$ in N_2 that sets the links correctly, as follows. Let $S \subseteq \Sigma$ be the set of object types that occur in R.
- The sequence $M_\emptyset \xrightarrow{*} M'_b$ creates objects whose type occurs in R. To that end, it uses transitions $t_i \in T_i$ to obtain a marking M'_b where for all places $p_b \in P_b$ with $\mathrm{color}_2(p_b) = \langle \sigma \rangle$ for $\sigma \in S$, and the unique place $p \in P_\blacktriangleright$ such that $\mathrm{color}_2(p_b) = \mathrm{color}_2(p)$, it holds that $M_b(p_b) = M_0(p)$ because $F_{out,2}(t_i, p_b) = \langle \nu_\sigma \rangle$.
- The sequence $M'_b \xrightarrow{*} M'_a$ uses only transitions $t_{\{\sigma_m,\sigma_o\}} \in T_L$, with the aim to populate P_L with the required links. For every relationship $(\sigma_m, \sigma_o) \in R$, there exist two places $p_{\sigma_m,(\sigma_m,\sigma_o)}, p_{\sigma_o,(\sigma_m,\sigma_o)} \in P_b$, one transition $t_{(\sigma_m,\sigma_o)} \in T_L$, two places $p_{(\sigma_m,\sigma_o),\sigma_m}, p_{(\sigma_m,\sigma_o),\sigma_o} \in P_a$, and one place $p_{(\sigma_m,\sigma_o)} \in P_L$ that satisfy $F_{in,2}(p_{\sigma_m,(\sigma_m,\sigma_o)}, t_{(\sigma_m,\sigma_o)}) = F_{out,2}(t_{(\sigma_m,\sigma_o)}, p_{\sigma_m,(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m} \rangle$ (resp. $F_{in,2}(p_{\sigma_o,(\sigma_m,\sigma_o)}, t_{(\sigma_m,\sigma_o)}) = F_{out,2}(t_{(\sigma_m,\sigma_o)}, p_{\sigma_o,(\sigma_m,\sigma_o)}) = \langle x_{\sigma_o} \rangle$), as well as $F_{out,2}(t_{\{\sigma_m,\sigma_o\}}, p_{(\sigma_m,\sigma_o)}) = \langle X_{\sigma_m}, x_{\sigma_o} \rangle$. For every object o, if $ot(o) = \sigma_m$ (resp. $ot(o) = \sigma_o$), then $\langle o \rangle \in M'_b(p_{\sigma_m,(\sigma_m,\sigma_o)})$ (resp. $\langle o \rangle \in M'_b(p_{\sigma_m,(\sigma_m,\sigma_o)})$).
  For every transition $t_{\{\sigma_m,\sigma_o\}} \in T_L$ and every object $o_o \in \mathcal{O}$ with $ot(o_o) = \sigma_o$, a binding β_{o_o,σ_m} can be selected that creates all links of this type that are required later. Let O_m be the set of all objects of type σ_m that occur with o_o in the run of N_1, i.e., all objects $o_m \in \mathcal{O}$ such that $\mathrm{type}(o_m) = \sigma_m$ and $\{o_o, o_m\} \subseteq \mathit{codom}(\beta_i)$. Then $\beta_{o_o,\sigma_m}(x_{\sigma_o}) = o_o$ and $\beta_{o_o,\sigma_m}(X_{\sigma_m}) = [O_m]$. As all required objects are present in the according places of P_b, $(t_{\{\sigma_m,\sigma_o\}}, \beta_{o_o,\sigma_m})$ can fire. Every object o_o and $o_m \in O_m$ is only considered once by a transition $t_{\{\sigma_m,\sigma_o\}}$, as $F_{out,2}(t_{\{\sigma_m,\sigma_o\}}, p_{\sigma_m,(\sigma_m,\sigma_o)}) = F_{out,2}(t_{\{\sigma_m,\sigma_o\}}, p_{\sigma_o,(\sigma_m,\sigma_o)}) = \bot$. Because the run in N_1 satisfies the relationship $(\sigma_m, \sigma_o) \in R$, for all bindings β_i it holds that if $o_m \in \beta_i(x_{\sigma_m})$ and an object $o'_o \in \beta_i(x_{\sigma_m})$ of type $\mathrm{type}(o_o) = \sigma_o$, then $o'_o = o_o$. Therefore, for all $p_{(\sigma_m,\sigma_o)} \in P_L$ it holds that for every pair of links

$(o_m, o_o), (o'_m, o'_o) \in M'_a(p_{(\sigma_m,\sigma_o)})$ if $o_m = o'_m$ then $o_o = o'_o$. All links in the places P_L are set correctly for the run in N_1.
For all transitions and bindings (t_i, β_i) in the run, the marking of all $p_{(\sigma_m,\sigma_o)} \in P_L$ is not changed as $F_{in,2}(p_{(\sigma_m,\sigma_o)}, t) = F_{out,2}(t, p_{(\sigma_m,\sigma_o)})$ for all $t \in T \setminus (T_L \cup T_c)$. All relationships are stable.

– The sequence $M'_a \xrightarrow{*} M'_0$ fires transitions to populate $P_\blacktriangleright$ for every object o in the run: Let $ot(o) = \sigma$. First, if $\sigma \notin S$, there is a flow from the according $t_\sigma \in T_e$ to a place $p \in P_\blacktriangleright$ with $F_{out,2}(t_\sigma, p) = \langle \nu_\sigma \rangle$. Second, if $ot(o) \in S$, there is a place $p_a \in P_a$ with $\text{color}_2(p_a) = \langle \sigma \rangle$ that holds a token $\langle o \rangle$. Hence, a binding $\beta(x_\sigma) = o$ exists for all $F_{in,2}(p_a, t_e) = \langle x_\sigma \rangle$, resulting in a marking where $\langle o \rangle \in M'_0(p)$ because $F_{out,2}(t_e, p) = \langle x_\sigma \rangle$.

Next we show the following $(\star)$: For all $1 \leq i \leq n$, a transition firing $M_{i-1} \xrightarrow{(t_i,\beta_i)} M_i$ where for all $p \in P_1$ it holds that $M_{i-1}(p) = M'_{i-1}(p)$, implies the existence of a firing $M'_{i-1} \xrightarrow{(t_i,\beta_i)} M'_i$, such that that $M_i(p) = M'_i(p)$. For all transitions $t \in T_1 \setminus (T_e \cup T_c) = T_2 \setminus (T_i \cup T_L \cup T_e \cup T_c)$ and all places $p \in P_2 \setminus (P_L \cup P_b \cup P_a) = P_1$, it holds that all arc inscriptions are singleton tuples such that $F_{out,2}(t, p) = F_{out,1}(t, p)$, $F_{in,2}(p, t) = F_{in,1}(p, t)$. If the binding β_i considers objects $o_m \in O_m$ with $ot(o_m) = \sigma_m$ and o_o with $ot(o_o) = \sigma_o$ of two types that are in a relationship $(\sigma_m, \sigma_o) \in R$, there is a place $p_{\sigma_m,\sigma_o} \in P_L \subset P_2$ where $O_m \times \{o_o\} \subseteq M'_0(p_{\sigma_m,\sigma_o})$. Because the relationship is stable, we also have $O_m \times \{o_o\} \subseteq M'_{i-1}(p_{\sigma_m,\sigma_o})$. Therefore, for the binding β_i from the run of N_1 can also be used in the run of N_2. As inflow and outflow are equal for all t_i and places $p \in (P_1 \cup P_L) \subset P_2$, for the resulting marking satisfies $M_i(p) = M'_i(p)$.
In M_n, only the places $P_\blacksquare$ hold tokens, such that $M_n(p) = \emptyset$ for all $p \in P_1 \setminus P_\blacksquare$. Otherwise, no transition $t \in T_c \subset T_1$ could remove the remaining tokens from any place $p \in P_1 \setminus P_\blacksquare$, because $F_{in}(p, t) = \perp$, and $M_\emptyset$ could not be reached. For the collapsing sequence $M_n \xrightarrow{*} M_\emptyset$, there exists an equal collapsing sequence $M'_n \xrightarrow{*} M_\emptyset$ of transitions $t \in T_e \subset T_2$. For every object $o_m \in \mathcal{O}$ of type $\text{type}(o_m) = \sigma_m$ and every relationship (σ_m, σ_o), there exists a place $p_{\sigma_m,\sigma_o} \in P_L$ with exactly one token $(o_m, o_o) \in M'_n(p_{\sigma_m,\sigma_o})$. There exists a final place $p \in P_\blacksquare$ with $o_m \in M'_n(p)$, and $F_{in,2}(p, t_e) = \langle x_{\sigma_m} \rangle$ and the link place with $F_{in,2}(p_{\sigma_m,\sigma_o}, t_e) = \langle x_{\sigma_m}, x_{\sigma_o} \rangle$. Hence, a binding $\beta'(\langle x_{\sigma_m}, x_{\sigma_o} \rangle) = \{o_m, o_o\}$ exists for every o_m and is enabled. Therefore, all objects can be removed with their many-to-one links.
A simple inductive argument using $(\star)$, and the observations about the emitting and link creating run, and the collapsing run, shows shows that the total run (4) is possible.

(3) TBD

□